

NEXUS YIĞINI · LİB / NEXUSMODEL

NexusModel Kullanım Kılavuzu

Katmanlar, ileri geçiş, geri yayılım, serileştirme ve ekosistemle eğitim.
API'ler, dosya haritası, formüller ve çalışan örnekler.

Sürüm 1.0 · C++20 · host float32

MatrixFlash Pro · NexusData · NexusLoss · NexusOptim

Autograd kararı: Yol B — analitik backward + katman-lokal MicroTape

00 İindekiler

- 01 NexusModel nedir
- 02 Ekosistem ve bir eęitim adımı
- 03 Autograd kararı
- 04 Kurulum ve CMake
- 05 Beş dakikada ilk model
- 06 Eęitim sözleşmesi
- 07 Tensor
- 08 Parameter ve başlatma
- 09 Module
- 10 Serileştirme
- 11 Konteynerler
- 12 Linear
- 13 Aktivasyonlar
- 14 Dropout
- 15 Konvolüsyon
- 16 Pooling
- 17 Normalizasyon
- 18 Embedding
- 19 RNN, LSTM, GRU
- 20 Attention ve Transformer
- 21 Flatten, Reshape, Residual
- 22 MicroTape
- 23 SIMD, CUDA, MatrixFlash
- 24 Uçtan uca eęitim
- 25 Performans
- 26 Testler ve ölçüm
- 27 Dosya haritası
- 28 Sınırlar ve hata mesajları

01 NexusModel nedir

NexusModel, bir sinir ağı *mimarisi* kütüphanesidir. Katmanı tanımlar, katmanları birbirine bağlar, ileri geçişi çalıştırır ve kayıptan gelen gradyanı katman katman geri yayar. Öğrenilen ağırlıkların gradyanını `Parameter::grad` içinde biriktirir. Ağırlığı kendisi güncellemez.

Tek genel başlık:

```
#include "nexus_model/nexus_model.hpp"
using namespace nexus_model;
```

Ad alanı `nexus_model` 'dir. Tensör tipi `nexus_model::Tensor` 'dır. Bu, MatrixFlash'ın `matrix_pro::Tensor` tipi değildir. İkisi de satır-major float32 taşır; NexusModel'in tensörü eğitim sıcak yolu için 64 bayt hizalı host belleğidir.

Ne yapar

- Katman: Linear, Conv1D/2D/3D, DepthwiseConv2D, pooling, BatchNorm, LayerNorm, GroupNorm, InstanceNorm, Dropout, Embedding, RNN, LSTM, GRU, MultiHeadAttention, Transformer encoder/decoder, pozisyon kodlama, Flatten, Reshape, Residual.
- Kap: Sequential, ModuleList, ModuleDict.
- Parametre listesi, `train()` / `eval()`, `state_dict` / `load_state_dict`.
- Kaiming, Xavier, orthogonal ve temel başlatıcılar.

Ne yapmaz

- Kayıp hesabı. Bu NexusLoss'un işidir. Çapraz entropi burada yoktur.
- Ağırlık adımı. SGD ve AdamW NexusOptim'dedir.
- Veri yükleme, karıştırma, bölme. Bunlar NexusData'dadır.
- Genel matris cebri ve cuBLAS. CPU GEMM bu kütüphanededir çünkü MatrixFlash `multiply` yalnızca CUDA'dır. GPU çarpımı istenirse MatrixFlash'e bırakılır.
- Epoch döngüsü, log ve checkpoint zamanlaması. Örnek eğitim döngüsü vardır; kalıcı orkestrasyon NexusTrain'e aittir.

Altın kural

NexusModel hangi hesabın hangi sırayla yapılacağını ve gradyanın nasıl akacağını bilir. Toplama, çarpma ve aktivasyonun makine kodu `kernel`s tablosundadır. Kayıp gradyanının üretilmesi çağırının işidir.

02 Ekosistem ve bir eğitim adımı

Dört kütüphane aynı teybi paylaşmaz. Veri `nexusdata::NDArray`, kayıp `std::span / std::vector`, optimizer ödünç `float*`, model ise `nexus_model::Tensor` kullanır. Eğitim adımı bu dört tipi elle birleştirir.

Kütüphane	Klasör	Sorumluluk
MatrixFlash Pro	matrixflash-pro	Cihaz tensörü, CUDA GEMM, sınırlı autograd
NexusData	NexusData	Dataset, DataLoader, NDArray batch
NexusModel	NexusModel	Katman, forward, backward, parametre gradyanı
NexusLoss	NexusLoss	Kayıp ve dL/d(tahmin)
NexusOptim	NexusOptim	Parametre güncellemesi

Bir adımın sırası sabittir:

1. `optimizer.zero_grad()` veya `model.zero_grad()` — parametre gradyanını sıfırla. Backward biriktirir, sıfırlamaz.
2. Batch'i `Tensor`'a kopyala. `Batch.inputs` bir `NDArray`'dir.
3. `prediction = model.forward(inputs)`.
4. NexusLoss ile satır satır kayıp ve `dL/dy` üret. Ortalama için batch boyutuna böl.
5. `model.backward(grad)` — gradyan ilk katmana kadar iner, `Parameter::grad` dolar.
6. `optimizer.step()` — NexusOptim aynı `float*` tamponlarına yazar.

İşaretçiler sabit kalmalı

`as_param_group` ağırlık ve gradyan işaretçilerini ödünç verir. Optimizer yaşarken `Parameter::data` ve `Parameter::grad` yeniden ayrılmamalıdır. `zero_grad` boyutu değişmediyse aynı tamponu sıfırlar.

03 Autograd kararı

MatrixFlash Pro'da ters-mod autograd vardır: `matrix_pro::Variable` ve sınırlı `matrix_pro::VarTensor`.

`Variable::backward()` kök gradyanı birlerle doldurur. NexusLoss'un ürettiği dış `dL/dy` vektörünü bu çağrıya veremezsiniz. NexusData ve NexusOptim de bu grafiğe bağlı değildir.

Bu yüzden Yol A seçilmedi. Yol A'da katman yalnızca forward yazar, global teyp türev alırdı. NexusModel **Yol B** kullanır:

1. **Analitik backward.** Linear, Conv, Pool, Norm, Dropout, Embedding, RNN, LSTM, GRU ve aktivasyonlar kendi türevini yazar. Parametre gradyanı doğrudan `Parameter::grad` içine `+=` ile birikir.
2. **MicroTape.** Yalnızca attention içindeki softmax o matmul zinciri küçük, katman-lokal bir teyp yazılır. Teyp o katmanın forward'ı kadar yaşar. Global bir autograd motoru değildir. LSTM ve GRU teyp yazılmaz; analitik BPTT kullanılır.

Ayrıntı dosyası: `include/nexus_model/core/details.txt`.

Neden analitik yol daha hızlı

Sıcak katmanın türevi birkaç GEMM ve biriktirmedir. Genel teyp her ara tensörü saklar ve sanal çağrı yapar. Linear ve Conv için formül literatürde kapalıdır; hata, sonlu fark testiyle yakalanır.

04 Kurulum ve CMake

Derleyici C++20 ister. Windows'ta denenen araç zinciri Visual Studio 2022 (MSVC 19.44), /W4 /WX /O2. GCC ve Clang tarafında uyarılar hata sayılır; Release'de -O3; ISA bayrakları yalnız ilgili kaynak dosyalarında.

```
cmake -S NexusModel -B NexusModel/build
cmake --build NexusModel/build --config Release
NexusModel\build\Release\nexus_model_tests.exe
NexusModel\build\Release\nexus_model_bench.exe
```

Çekirdek hedef `nexus_model` statik kütüphanedir. Include dizini `NexusModel/include`. Başka bir CMake projesine:

```
add_subdirectory(path/to/NexusModel)
target_link_libraries(uygulama PRIVATE nexus_model)
```

Seçenekler

Seçenek	Varsayılan	Anlamı
NEXUS_MODEL_BUILD_TESTS	ON	<code>nexus_model_tests</code>
NEXUS_MODEL_BUILD_BENCHMARKS	ON	<code>nexus_model_bench</code>
NEXUS_MODEL_BUILD_EXAMPLES	OFF	NexusData, NexusLoss, NexusOptim yalnızca örneğe bağlanır
NEXUS_MODEL_ENABLE_AVX512	ON	AVX-512 çeviri birimi. Çalışma zamanında CPU yoksa kullanılmaz
NEXUS_MODEL_WITH_CUDA	OFF	Fused softmax, LayerNorm, ReLU, GELU cihaz çekirdekleri
NEXUS_MODEL_WITH_MATRIXFLASH	OFF	İsteğe bağlı GPU GEMM köprü başlığı
NEXUS_MODEL_WARNINGS_AS_ERRORS	ON	Uyarı = hata

```
cmake -S NexusModel -B NexusModel/build-e2e -DNEXUS_MODEL_BUILD_EXAMPLES=ON
cmake --build NexusModel/build-e2e --config Release --target train_end_to_end
```

Örnek hedefi `nexus_model`, `nexusloss`, `nexus_optim` ve `nexusdata` bağlar. Çekirdek `.lib` bu üçüne bağlı değildir. `as_param_group` başlığı da çekirdeğe dahil değildir; yalnızca örneği derleyen dosya onu include eder.

05 Beş dakikada ilk model

Aşağıdaki program NexusData olmadan çalışır. Girdi elle doldurulur, kayıp burada yazılmaz; geri yayılımı görmek için çıkış gradyanı birlerle doldurulur. Bu, "her logit artsın" anlamına gelir ve gerçek eğitim değildir. Gerçek adım 24. bölümdedir.

```
#include "nexus_model/nexus_model.hpp"
#include <iostream>

int main() {
    using namespace nexus_model;
    Sequential net({
        std::make_shared<Linear>(4, 8),
        std::make_shared<ReLU>(),
        std::make_shared<Linear>(8, 3)
    });

    Tensor x = Tensor::from_values({2, 4}, {
        0.2f, -0.1f, 0.5f, 0.0f,
        0.3f, 0.4f, -0.2f, 0.1f
    });
    Tensor y = net.forward(x);          // şekil [2, 3]
    Tensor gy = Tensor::full({2, 3}, 1.f);
    net.zero_grad();
    Tensor dx = net.backward(gy);       // şekil [2, 4]

    std::cout << "cikis " << y.numel()
               << " giris gradyani " << dx.numel()
               << " parametre " << net.parameters().size() << "\n";
}
```

`parameters().size()` burada 4'tür: iki ağırlık ve iki bias. ReLU parametre taşımaz.

Şekil sözleşmesi

`Linear(in, out)` son boyutu `in` olan her tensörü kabul eder. `[2, 4]` girer, `[2, 3]` çıkar. `[8, 5, 4]` de geçerlidir; çıkış `[8, 5, 3]` olur. Satırlar GEMM'de düzleştirilir.

06 Eğitim sözleşmesi

PyTorch'taki `loss.backward()` burada iki çağrıya bölünmüştür. NexusLoss skaler kaybın tahmine göre türevini üretir. NexusModel o vektörü ağa yayar.

```
model.zero_grad();
Tensor y = model.forward(x);
// grad, y ile aynı şekilde, dL/dy
model.backward(grad);
optimizer.step();
```

Çağrı	Kim sıfırlar	Ne birikir
forward	katman kendi çıkış tamponunu yeniden kullanır	ara değer (giriş kopyası, maske, running stat)
backward	giriş gradyanı her seferinde sıfırlanır	<code>Parameter::grad</code> sıfırlanmaz, eklenir
zero_grad	çağırın, adım başında	tüm parametre gradyanları

zero_grad atlanırsa

İkinci adımın gradyanı birincinin üstüne biner. Model sessizce yanlış yönde öğrenir. Her adımın başında `model.zero_grad()` veya optimizer'ın `zero_grad()` çağrısı gerekir. İkisi aynı tamponu sıfırlar.

`train()` Dropout'u maskeli, BatchNorm'u batch istatistiğiyle çalıştırır. `eval()` Dropout'u birim yapar, BatchNorm running mean ve running var kullanır. Varsayılan mod eğitimidir. `eval()` çocuklara da iner.

Dönen `Tensor` depolamayı paylaşan bir başlıktır. Veri kopyası değildir. Sonraki `forward`, önceki çıkışı hâlâ tutuyorsanız yeni tampon ayırır. Eğitim döngüsünde tahmini kayıp hesabı bitince bırakın.

07 Tensor

Dosya: `include/nexus_model/core/tensor.hpp`. Rank en fazla 8'dir (`kMaxRank`). Boyutlar nesnenin içinde durur, şekil için `std::vector` zorunlu değildir. Veri `shared_ptr` ile paylaşılır. Windows'ta ayırma `_aligned_malloc`, hiza 64 bayttır.

API	Ne yapar
<code>Tensor::zeros({n, m})</code>	Sıfırlanmış sahip tensör
<code>Tensor::full(shape, v)</code>	Her eleman v
<code>Tensor::from_values(shape, {...})</code>	Eleman sayısı şekille uyuşmazsa <code>ModelError</code>
<code>Tensor::borrow(ptr, shape)</code>	Sahip değildir. İşaretçinin ömrü çağırandadır
<code>rank()</code> , <code>numel()</code> , <code>size(axis)</code> , <code>dims()</code> , <code>shape_vec()</code>	Şekil sorgusu
<code>data()</code> , <code>operator[]</code>	Düz satır-major erişim
<code>zero()</code> , <code>fill(v)</code> , <code>copy_from</code> , <code>clone</code>	clone yeni depolama ayırır
<code>resize_dims(ptr, rank)</code> , <code>resize_like</code>	Tek sahip ve kapasite yetiyorsa ayırmaz
<code>reshape({2, -1})</code> , <code>view_as</code>	Aynı depolama, yeni şekil. Eleman sayısı aynı olmalı
<code>allocation_count()</code>	AlignedStorage kurucu sayacı. Sıcak yol testinde kullanılır

```
Tensor a = Tensor::from_values({2, 3}, {1, 2, 3, 4, 5, 6});
Tensor row = a.reshape({2, 3});    // paylaşır
Tensor copy = a.clone();           // bağımsız
a[0] = 9.f;                        // row[0] da 9, copy[0] hâlâ 1
```

Düzen satır-majordur. `[N, C, H, W]` için `W` en hızlı değişen indekstir. `[B, T, F]` için zaman adımı bitişik değildir; RNN zaman dilimini paketlerken buna dikkat eder.

08 Parameter ve başlatma

Dosya: `core/parameter.hpp`, `core/initializer.hpp`. Ad alanı `nexus_model::init`.

```
class Parameter {
public:
    Tensor data;
    Tensor grad;
    bool requires_grad;
    explicit Parameter(Tensor values, bool need_grad = true);
    void zero_grad();
    void accumulate_grad_locked(const Tensor& extra);
};
```

Kurucu `grad`'ı `data` ile aynı şekilde sıfırlar. `requires_grad == false` ise backward o tampona yazmaz. `accumulate_grad_locked` sıcak yolda değildir; veri paralel kopyaların gradyanını adım dışında birleştirmek içindir. Aynı modül nesnesinde eşzamanlı `forward` tanımsızdır, çünkü ara tamponlar paylaşılır.

İşlev	Kullanım
<code>manual_seed(uint64)</code>	İş parçacığı yerel mt19937. Varsayılan tohum 0xC0FFEE
<code>zeros_ / ones_ / constant_(t, v)</code>	Bias çoğu katmanda sıfır
<code>uniform_(t, low, high)</code>	Kapalı aralık
<code>normal_(t, mean, std)</code>	Embedding varsayılanı normal(0, 1)
<code>kaiming_uniform_(t, slope=0, fan_out=false)</code>	Linear ve Conv varsayılanı. He, 2015
<code>kaiming_normal_</code>	Aynı kazanç, normal dağılım
<code>xavier_uniform_(t, gain=1)</code>	Glorot. Sigmoid ve tanh katmanları
<code>xavier_normal_</code>	Aynı sınır, normal
<code>orthogonal_(t, gain=1)</code>	RNN tekrarlayan ağırlık. Saxe, 2014

Kaiming uniform sınırı $\text{bound} = \sqrt{3} \cdot \sqrt{(2 / (1+a^2))} / \sqrt{\text{fan}}$ `fan_in` varsayılandır. Conv'de `fan`, kanal çarpı alıcısı alandır. Satır sayısı sütundan küçükse orthogonal, devriğini çarpanlara ayırır.

```
Linear layer(32, 16);
init::xavier_uniform_(layer.weight()->data); // varsayılan Kaiming'i ezer
init::zeros_(layer.bias()->data);
```

09 Module

Dosya: `core/module.hpp` . Bütün katmanlar bundan türer.

Üye	Sözleşme
<code>forward(input) → Tensor</code>	Çıkış. Depolama katmana aittir
<code>backward(grad_output) → Tensor</code>	Giriş gradyanı. Parametre gradyanı birikir
<code>parameters()</code>	Bu modül ve çocuklar. Yalnız <code>requires_grad</code> filtresi uygulamaz; filtre <code>as_param_group</code> içindedir
<code>named_parameters(prefix)</code>	<code>"0.weight"</code> , <code>"attn.q_proj.bias"</code>
<code>zero_grad()</code>	Ağaçtaki her parametre
<code>train() / eval() / is_training()</code>	Çocuklara yayılır
<code>state_dict()</code>	Parametre ve buffer klonu
<code>load_state_dict(state, strict=true)</code>	strict iken fazla veya eksik anahtar <code>ModelError</code>

Alt sınıf çocuk bağlamak için korumalı `register_parameter` , `register_buffer` , `register_module` kullanır. Buffer öğrenilmez ama checkpoint'e girer. BatchNorm'un `running_mean` ve `running_var` buffer'dır.

Hata tipi `nexus_model::ModelError` , `std::runtime_error` türevidir. Mesajlar İngilizcedir: `"Linear input last dimension must equal in_features"` gibi.

10 Serileştirme

Dosya: `core/serialization.hpp`. `StateDict` bir `std::map<string, Tensor>` tutar. `state_dict()` değerleri klonlar; sonra eğitim checkpoint'ini bozmaz.

```
StateDict snap = model.state_dict();
save_state_dict(snap, "model.nxm");
StateDict loaded = load_state_dict_file("model.nxm");
model.load_state_dict(loaded);
```

İkili düzen, küçük endian:

1. 4 bayt sihir: `NXM1`
2. `uint32` kayıt sayısı
3. Her kayıt: `uint32` isim uzunluğu, isim baytları, `uint32` rank, rank adet `uint64` boyut, ham float32

Anahtarlar hiyerarşiktir. `Sequential` çocuklarını "0", "1" diye adlandırır. Linear'ın alanı `weight` ve varsa `bias` olur. Üç katmanlı bir ağda ilk ağırlık `0.weight`'tir.

```
bool threw = false;
try {
    StateDict extra = model.state_dict();
    extra.items["yok"] = Tensor::zeros({1});
    model.load_state_dict(extra);           // strict
} catch (const ModelError&) { threw = true; }
```

`load_state_dict(state, false)` eksik anahtar atlar, fazla anahtar yok sayar. Şekil uyumsuzluğu strict olmasa da hata verir.

11 Konteynerler

Sequential

`containers/sequential.hpp`. Forward sırayla, backward tersten. Ara aktivasyonu ikinci kez saklamaz; her katman kendi girdisini tutar.

```
auto net = Sequential({
    std::make_shared<Linear>(784, 256),
    std::make_shared<ReLU>(),
    std::make_shared<Dropout>(0.3f),
    std::make_shared<Linear>(256, 10)
});
net.add(std::make_shared<LogSoftmax>());
Module& first = net[0];
```

`size()` katman sayısıdır. İsimler ekleme sırasının metnidir, bu yüzden araya katman sokmak eski checkpoint anahtarlarını kaydırır.

ModuleList ve ModuleDict

İkisi de `parameters()` için çocukları kaydeder. İkisinin de örtük `forward` 'u yoktur; çağırırsanız `ModelError` fırlatır. Sıra veya isim sizdedir.

```
ModuleList trunk({ std::make_shared<Linear>(32, 32) });
ModuleDict heads;
heads.add("sinif", std::make_shared<Linear>(32, 10));
heads.add("kutu", std::make_shared<Linear>(32, 4));
Tensor h = trunk[0].forward(x);
Tensor logits = heads["sinif"]->forward(h);
```

Geri yayılımı da sizin sıranız belirler. İki başlık aynı gövdeyi paylaşıyorsa gövde gradyanı iki backward'ın toplamıdır; arada `zero_grad` çağırma.

12 Linear

Dosya: `layers/linear.hpp`.

`Linear(in_features, out_features, use_bias = true)`

$$Y = X W^T + b \quad X: [..., in] \quad W: [out, in] \quad b: [out]$$

$$dX += dY \cdot W \quad dW += dY^T \cdot X \quad db += \Sigma dY$$

Çekirdek biriktirir. `dx` her backward'da sıfırlanır. `dW` ve `db` ancak `zero_grad` ile temizlenir. Erişim, devrik matrisi belleğe yazmadan satır satır axpy'dir.

```
Linear fc(2, 3, true);
float* w = fc.weight()->data.data();
// W satırları: [1, 0], [0, 1], [1, 1]
w[0]=1; w[1]=0; w[2]=0; w[3]=1; w[4]=1; w[5]=1;
fc.bias()->data[0]=0; fc.bias()->data[1]=0; fc.bias()->data[2]=1;
Tensor x = Tensor::from_values({2, 2}, {1, 2, 3, 4});
Tensor y = fc.forward(x);
// y = {1, 2, 4, 3, 4, 8}
```

`weight()` ve `bias()` `shared_ptr<Parameter>` döner. `in_features == 0` veya çıkış 0 ise kurucu hata verir. Son boyut `in_features` değilse forward hata verir.

13 Aktivasyonlar

Dosya: `activations/activations.hpp`. Hepsi `Module` 'dür, Sequential'a girer. ReLU, LeakyReLU, GELU'nun tanh yaklaşımı, Sigmoid, Tanh, SiLU ve Softmax AVX2 tablosuna bağlıdır. Tam GELU (erf) skalar kalır.

Sınıf	İleri	Geri
ReLU()	$\max(x, 0)$	$x > 0$ iken dy , değilse 0. $x = 0$ alttürevi 0
LeakyReLU($\alpha=0.01$)	$x > 0 ? x : \alpha x$	eğim 1 veya α
PReLU($\text{init}=0.25$)	öğrenilen α	α da gradyan alır
ELU($\alpha=1$)	$x > 0 ? x : \alpha(e^x - 1)$	Clevert, 2016
GELU($\text{tanh_approx}=\text{false}$)	$x \Phi(x)$	Hendrycks. true ise tanh yaklaşımı
Sigmoid()	$1/(1+e^{-x})$	$\sigma(1-\sigma)$
Tanh()	$\tanh$	$1 - \tanh^2$
SiLU()	$x \sigma(x)$	Elfwing, 2018
Mish()	$x \tanh(\text{softplus}(x))$	Misra, 2019
Softmax()	son boyutta, max çıkarılır	$y \odot (dy - \langle dy, y \rangle)$
LogSoftmax()	$x - \text{logsumexp}$	$dy - \text{softmax} \cdot \Sigma dy$

Softmax ve çapraz entropi

NexusLoss `CrossEntropyLoss` ham logit ister. Modele ayrıca Softmax koyup onun çıkışını bu kayba vermeyin. LogSoftmax da aynı kayıpla çift sayılır. Sınıflandırma kafası Linear ile bitsin.

```
Sequential block({
  std::make_shared<Linear>(64, 64),
  std::make_shared<GELU>(true), // tanh yaklaşımı, SIMD
  std::make_shared<Linear>(64, 10)
});
```

14 Dropout

`Dropout(probability = 0.5, seed = 0xD1CE)`

Eğitimde ters dropout: $y = x \cdot \text{mask} / (1 - p)$ $\text{mask} \sim \text{Bernoulli}(1 - p)$ Değerlendirmede $y = x$. Backward aynı maskeyi kullanır: $dx = dy \cdot \text{mask} / (1 - p)$

Maske forward anındaki `is_training()` ile seçilir ve saklanır. $p = 0$ birimdir. $p = 1$ eğitimde çıkışı sıfırlar. Tohum her forward'da artar; aynı tohumla kurulan iki katman, aynı uzunluktaki ilk çağrıda aynı maskeyi üretir.

```
Dropout drop(0.3f, 7);
drop.train();
Tensor y = drop.forward(x);
drop.eval();
Tensor z = drop.forward(x); // z, x ile aynı
```

15 Konvolüsyon

Dosya: `layers/conv.hpp`. PyTorch ile aynı çapraz korelasyon. Çekirdek `im2col` ile açılır, çarpım `Linear` ile aynı GEMM'e gider. Workspace katmanda bir kez ayrılır: sütun tamponu ve onun gradyanı, yani $2 \times \text{col}$.

Girdi `[N, C, uzamsal...]`. Ağırlık `[K, C/groups, çekirdek...]`. Uzamsal boyutlar sağa hizalanır; `Conv1D`, `Conv2D` ve `Conv3D` aynı çekirdeği paylaşır.

Kurucu	Not
<code>Conv1D(in, out, k, stride=1, pad=0, dil=1, groups=1, bias=true)</code>	Girdi <code>[N,C,L]</code>
<code>Conv2D(in, out, k, stride=1, pad=0, dil=1, groups=1, bias=true)</code>	Kare çekirdek. Girdi <code>[N,C,H,W]</code>
<code>Conv2D(in, out, kH, kW, sH, sW, pH, pW, dH, dW, groups=1, bias=true)</code>	Ayrı eksen
<code>Conv3D(in, out, k, stride=1, pad=0, dil=1, groups=1, bias=true)</code>	Üç eksen aynı sayılar. Girdi <code>[N,C,D,H,W]</code>
<code>DepthwiseConv2D(channels, k, stride=1, pad=0)</code>	<code>groups = channels</code> , çıkış kanalı = giriş

$$\text{çıkış} = \lfloor (\text{girdi} + 2 \cdot \text{pad} - \text{dil} \cdot (\text{çekirdek} - 1) - 1) / \text{stride} \rfloor + 1$$

Boş çıkış `ModelError` verir. Groups, `in_channels`'ı bölmek zorundadır.

```
Conv2D conv(1, 1, 2, 1, 0);          // 2x2, stride 1, pad 0
float* w = conv.parameters()[0]->data.data();
w[0]=1; w[1]=0; w[2]=0; w[3]=1;      // birim köşegen
conv.parameters()[1]->data.zero();
Tensor img = Tensor::from_values({1,1,3,3}, {1,2,3, 4,5,6, 7,8,9});
Tensor y = conv.forward(img);        // {6, 8, 12, 14}, şekil [1,1,2,2]
```

Padding 1 ve 3×3 çekirdek, "same" boyut için stride 1 ile birlikte kullanılır: `Conv2D(16, 32, 3, 1, 1)`.

16 Pooling

Dosya: `layers/pooling.hpp`. Parametre yoktur.

Sınıf	İleri	Geri
<code>MaxPool2D(k)</code> veya <code>(kH,kW,sH,sW,pH,pW)</code>	Penceredeki ilk maksimum kazanır (>)	Gradyan yalnızca o indekse. İndeks forward'da saklanır
<code>AvgPool2D(k, stride=-1, pad=0, count_include_pad=true)</code>	Ortalama. stride verilmezse çekirdeğe eşittir	Pencereye eşit dağıtılır
<code>GlobalAvgPool2D()</code>	H·W üzerinden ortalama. Çıkış [N, C]	Her uzamsal elemana $1/(H \cdot W)$

```
MaxPool2D pool(2, 2);
Tensor x = Tensor::from_values({1,1,2,2}, {1.f, 3.f, 2.f, 0.f});
Tensor y = pool.forward(x);      // {3}
AvgPool2D avg(2, 2);
Tensor z = avg.forward(x);       // {1.5}
```

Sınıflandırma gövdesinden sonra tipik dizi: Conv → ReLU → Conv → ReLU → GlobalAvgPool2D → Linear. Global havuz uzamsal boyutu kaldırdığı için Linear'a [N, C] gider.

17 Normalizasyon

Dosya: `layers/normalization.hpp`. Kaynak: Ioffe ve Szegedy, 2015 (BatchNorm); Ba, Kiros ve Hinton (LayerNorm).

Sınıf	Eksen	Durum
<code>BatchNorm(num_features, eps=1e-5, momentum=0.1, affine=true)</code>	Kanal, N ve uzamsal birlikte	running_mean, running_var buffer
<code>BatchNorm1D</code> / <code>BatchNorm2D</code>	Aynı kurucu. 1D'de [N,C] veya [N,C,L], 2D'de [N,C,H,W]	Aynı
<code>LayerNorm(normalized_size, eps=1e-5)</code>	Son boyut	Öğrenilen weight=1, bias=0. Running yok
<code>GroupNorm(groups, channels, eps=1e-5, affine=true)</code>	Kanal grupları	Running yok
<code>InstanceNorm(channels, eps, affine)</code>	<code>GroupNorm(channels, channels)</code>	Her örnek, her kanal

Eğitimde BatchNorm batch ortalamasını ve popülasyon varyansını (bölüm) kullanır. `running_var` güncellemesi, $m > 1$ iken yansız bölendir (bölüm $m-1$). Momentum PyTorch'taki gibidir: $\text{running} \leftarrow (1 - \text{momentum}) \cdot \text{running} + \text{momentum} \cdot \text{batch}$. `eval()` saklanan running değerleri kullanır. Eval backward, eğitim Jakobyani değildir; yalnızca affine ölçek yoludur.

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} \quad y = \gamma \hat{x} + \beta$$

LayerNorm son boyutu `normalized_size` olmayan girdide hata verir. Transformer blokları bunu kullanır. BatchNorm küçük batch'te gürültülüdür; batch 1 ile eğitim istatistiği anlamsızlaşır, eval'e geçmek veya GroupNorm seçmek gerekir.

```
BatchNorm2D bn(32);
bn.train();
Tensor y = bn.forward(images); // running güncellenir
bn.eval();
Tensor z = bn.forward(images); // running kullanılır
```

18 Embedding

`Embedding(num_embeddings, embedding_dim, padding_idx = -1)`

Girdi float indekstir; değerler tam sayı olmalıdır. Çıkış, indeks tensörünün şekline bir boyut ekler: `[4] → [4, dim]`, `[B, T] → [B, T, dim]`. Backward yalnızca dokunulan satırlara scatter-add yapar. Aynı indeks iki kez geçtiyse o satırın gradyanı iki kez eklenir. `padding_idx` satırı 0 üretir ve gradyan almaz. Varsayılan `-1`, hiçbir satırı pad yapmaz. Başlangıç `normal(0, 1)`.

```
Embedding emb(1000, 64, /*padding_idx=*/0);
Tensor idx = Tensor::from_values({2, 3}, {1, 2, 0, 4, 4, 1});
Tensor y = emb.forward(idx);      // [2, 3, 64], 0. satırlar sıfır
```

19 RNN, LSTM, GRU

Dosya: `layers/recurrent.hpp`. Üçü de batch-first bekler: `[B, T, F]`. Çıkış `[B, T, H]`. Zaman dilimi bitişik olmadığı için her adım `[B, F]` olarak paketlenir. Hidden ve hücre tamponları kurucuda değil, ilk şekle göre katman alanında tutulur ve sonraki adımlarda üzerine yazılır.

Sınıf	Kapılar	Başlangıç
<code>RNN(input, hidden, layers=1, truncate_bptt=0)</code>	<code>tanh</code>	Kaiming + orthogonal tekrar
<code>LSTM(input, hidden, layers=1, truncate_bptt=0)</code>	<code>i, f, g, o</code> — PyTorch sırası	Forget bias bloğu 1
<code>GRU(input, hidden, layers=1, truncate_bptt=0)</code>	<code>r, z, n</code>	LSTM'den az parametre

$$\text{RNN: } h_t = \tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh})$$

$$\text{LSTM: } i, f, g, o = \text{parçala}(W x + U h + b) \quad c_t = \sigma(f) \odot c_{t-1} + \sigma(i) \odot \tanh(g) \quad h_t = \sigma(o) \odot \tanh(c_t)$$

$$\text{GRU: } r, z = \sigma(\dots) \quad n = \tanh(W_n x + r \odot U_n h) \quad h' = (1-z) \odot n + z \odot h$$

`truncate_bptt > 0` ise gradyan pencerenin öncesine gitmez. Uzun dizide bellek ve süre bununla kesilir. Çift yön ve paketli gizli durum dönüşü yoktur; her forward sıfır durumdan başlar, durum dizinin kendi zamanı boyunca taşınır.

```
LSTM lstm(32, 64, /*num_layers=*/1, /*truncate_bptt=*/0);
Tensor seq = Tensor::zeros({8, 20, 32}); // B=8, T=20, F=32
Tensor y = lstm.forward(seq); // [8, 20, 64]
```

Çok katmanlı kurulumda üst katmanın girdisi alt katmanın gizli dizisidir. Parametre adları katman indeksini taşır; checkpoint anahtarı buna göre uzar.

20 Attention ve Transformer

Dosya: `layers/attention.hpp`. Vaswani ve diğerleri, 2017. Q, K, V ve çıkış projeksiyonları analitik Linear'dır. Softmax ◦ matmul MicroTape'tedir.

$$\text{Attention}(Q,K,V) = \text{softmax}(Q K^T / \sqrt{d}) V$$

`MultiHeadAttention(embed, num_heads, dropout=0, causal=false)`

`embed` kafa sayısına bölünmelidir. Girdi `[B, T, E]`. Self-attention için `forward(x)` yeter. Çapraz attention:

```
Tensor ctx = mha.forward(query, key, value, /*mask=*/nullptr);
```

`causal=true` üst üçgene $-1e9$ ekler; query ve key uzunluğu eşit olmalıdır. Dropout, eğitimde ve $p > 0$ iken olasılıklara ters dropout uygular. Self-attention backward'ı $dq+dk+dv$ döner. Çapraz attention dq döner; bellek gradyanı `grad_key_input()` ve `grad_value_input()` içindedir. K ve V projeksiyonlarının parametre gradyanı her iki durumda da birikir.

`PositionalEncoding(embed, max_len=512, learnable=false)`

Sinüs ve kosinüs tablosu. `learnable=true` ise tablo parametre olur. Girdi `[B, T, E]`, T kurucudaki `max_len`'i aşmamalıdır.

`TransformerEncoderLayer(embed, heads, ff=0, dropout=0.1)`

`ff == 0` ise gizli genişlik `4*embed`. Post-norm:

$$x = \text{LN}(x + \text{Dropout}(\text{Attn}(x))) \quad x = \text{LN}(x + \text{Dropout}(\text{GELU-FFN}(x)))$$

`TransformerDecoderLayer(embed, heads, ff=0, dropout=0.1)`

Nedensel self-attention, ardından çapraz attention, ardından FFN. Üçü de post-norm. Tek argümanlı `forward(tgt)`, daha önce `forward(tgt, memory)` çağrılmadıysa hata verir. Dönüş, hedef gradyanıdır; bellek gradyanı çapraz attention üyesinin içinde kalır.

```
TransformerEncoderLayer enc(64, 8, /*ff=*/128, /*dropout=*/0.1f);
Tensor tokens = Tensor::zeros({2, 16, 64});
Tensor hidden = enc.forward(tokens);

TransformerDecoderLayer dec(64, 8, 128, 0.1f);
Tensor tgt = Tensor::zeros({2, 10, 64});
Tensor out = dec.forward(tgt, hidden);
```

Maske

`MultiHeadAttention::forward` dördüncü argümanı toplanan bir maskedir, çarpılan değil. Tutmak istemediğiniz konuma büyük negatif (örneğin $-1e9$) yazın. Decoder'ın self-attention'ı kendi nedensel maskesini kurar.

21 Flatten, Reshape, Residual

```
Flatten flat; // [N, ...] → [N, -1], kopyalar
Reshape view({-1, 32}); // eleman sayısı aynıysa görünüm
auto block = ResidualBlock(std::make_shared<Sequential>({
    std::make_shared<Linear>(32, 32),
    std::make_shared<ReLU>(),
    std::make_shared<Linear>(32, 32)
}));
```

Residual: $y = F(x) + x$ $dx = dF + dy$ $F(x)$ ile x 'in eleman sayısı aynı olmalıdır. Şekil denetimi yapmaz; uyumsuzluk taşar. Conv artığında kanal sayısı değişiyorsa F 'nin çıkışını x ile aynı şekle getirin, sonra ResidualBlock'a verin.

Flatten, çağırının deposuna görünüm döndürmez; kendi tamponuna kopyalar. Böylece sonraki katman girişi ezerse Flatten'ın sakladığı girdi bozulmaz.

22 MicroTape

Dosya: `core/micro_tape.hpp`. Kullanıcı kodu çoğu zaman buna dokunmaz. Attention onu içeriden kullanır. Yeni bir softmax-matmul katmanı yazıyorsanız aynı deseni izleyin.

İşlem	Şekil
<code>load(tensor)</code>	Yuvaya kopya. Kimlik döner
<code>matmul_bt(a, b)</code>	$[B,M,K] \times [B,N,K]^T \rightarrow [B,M,N]$
<code>matmul(a, b)</code>	$[B,M,K] \times [B,K,N] \rightarrow [B,M,N]$
<code>scale, add, mul</code>	Eleman bazında
<code>add_matrix(a, bias)</code>	$[R, \text{satır}, \text{sütun}]$ üzerine $[\text{satır}, \text{sütun}]$ yayınlanır. Bias'ın gradyanı yoktur
<code>softmax(a)</code>	Son boyut
<code>backward(root, grad)</code>	Yuvaların gradyanını sıfırlar, işlemleri tersten yürütür
<code>reset()</code>	İşlem listesini siler, kapasiteyi ve yuvaları tutar

İkinci forward, şekil aynıysa yuva ayırmaz. `value(id)` ve `grad(id)` o yuvaya bakar.

23 SIMD, CUDA, MatrixFlash

İşlemci

`simd/simd_dispatch.cpp` CUID ve `XGETBV` ile işletim sisteminin YMM/ZMM durumunu kaydedip kaydetmediğine bakar. AVX-512F ve OS desteği varsa `linear_forward`, `dot` ve `relu` AVX-512 seçimine gider; Linear bu girişten blokları AVX2 matris motoruna yönlendirir. Diğer sıcak çekirdekler AVX2+FMA'dır. Hiçbiri yoksa skalar gövde kalır.

```
nexus_model::kernels::set_force_scalar(true); // test ve karşılaştırma
auto y = layer.forward(x);
nexus_model::kernels::set_force_scalar(false);
```

AVX2 softmax: maksimum SIMD ile bulunur, sonra vektör exp (Cephes tarzı) ve toplam. Tam GELU erf skalar kalır.

CUDA

`NEXUS_MODEL_WITH_CUDA=ON` iken üç çekirdek derlenir. Host `Tensor` onları çağırır. Çağırılan cihaz işaretçisi verir:

İşlev	Dosya
<code>cuda::fused_softmax(x, y, rows, cols)</code>	<code>cuda/fused_softmax.cuh</code>
<code>cuda::fused_layernorm(x, gamma, beta, y, rows, cols, eps)</code>	<code>cuda/fused_layernorm.cuh</code>
<code>cuda::relu / cuda::gelu_tanh</code>	<code>cuda/activation_kernels.cuh</code>

MatrixFlash köprüsü

`bridge/matrixflash.hpp` yalnızca `NEXUS_MODEL_WITH_MATRIXFLASH` tanımlıyken `matrix_pro::multiply` sarmalar. Çıktı cihaz belleğindedir. NexusModel kendi cuBLAS'ını yazmaz. Varsayılan eğitim yolu host AVX GEMM'dir.

NexusOptim adaptörü

`adapters/nexus_optim.hpp` çekirdeğe bağlı değildir. Örneğe veya kendi eğitim dosyanıza ekleyin. `requires_grad == false` olanları atlar.

```
#include "nexus_model/adapters/nexus_optim.hpp"
#include "nexus_optim/algorithms/adamw.hpp"

nexus_optim::ParamGroupOptions opt;
opt.learning_rate = 1e-3;
auto group = nexus_model::as_param_group(model, opt);
nexus_optim::AdamW<float>::Options adam;
adam.lr = 1e-3;
adam.weight_decay = 0.01;
nexus_optim::AdamW<float> optimizer({group}, adam);
```

24 Uçtan uca eğitim

Dosya: `examples/train_end_to_end_example.cpp`. Aşağıdaki döngü o dosyanın sözleşmesidir ve bu makinede çalıştırılmıştır. 96 örnek, 8 özellik, 3 sınıf. Kayıp 8 epoch sonunda yaklaşık 1.68'den 0.084'e inmiştir.

Gerçek imzalar, tasarım taslağındaki sözde koddan farklıdır. Bunları kullanın:

- `nexusdata::InMemoryDataset(features [N,F], labels [N]). dataset.shuffle()` yoktur.
- `random_split(dataset, {0.8, 0.2}, seed)` bir `pair<Subset, Subset>` döner.
- `DataLoaderOptions` içinde `batch_size`, `shuffle`, `seed`.
- `Batch.inputs` ve `Batch.labels` `NDArray`'dir.
- `nexusloss::classification::CrossEntropyLoss<float>` tek örnek alır: `forward(span logits, size_t sınıf)`. Batch için satır döngüsü ve gradyanı batch boyutuna bölme gerekir. Umbrella başlık `nexusloss/nexusloss.hpp` regression'ı önce çektiği için onu include edin.
- AdamW, `AdamImpl<float, true>` kısayoludur.

```
#include "nexus_model/adapters/nexus_optim.hpp"
#include "nexus_model/nexus_model.hpp"
#include "nexusdata/dataset/in_memory.hpp"
#include "nexusdata/dataset/split.hpp"
#include "nexusdata/loading/dataloader.hpp"
#include "nexusloss/nexusloss.hpp"
#include "nexus_optim/algorithms/adamw.hpp"

auto model = nexus_model::Sequential({
    std::make_shared<nexus_model::Linear>(features, 16),
    std::make_shared<nexus_model::ReLU>(),
    std::make_shared<nexus_model::Linear>(16, classes)
});

nexusloss::classification::CrossEntropyLoss<float> loss_fn;
nexus_optim::AdamW<float> optimizer(
    { nexus_model::as_param_group(model, group_options) }, adam);

for (int epoch = 0; epoch < epochs; ++epoch) {
    for (nexusdata::Batch batch : loader) {
        optimizer.zero_grad();
        auto B = batch.inputs.shape()[0];
        nexus_model::Tensor inputs({B, features});
        // NDArray float32 → Tensor kopyası
        std::memcpy(inputs.data(), batch.inputs.data<float>(),
            inputs.numel() * sizeof(float));

        auto prediction = model.forward(inputs);
        auto grad = nexus_model::Tensor::zeros(prediction.shape_vec());
        for (size_t row = 0; row < B; ++row) {
            size_t label = (size_t)(batch.labels.data<float>()[row] + 0.5f);
            std::span<const float> logits(prediction.data() + row * classes, classes);
            loss_fn.forward(logits, label);
            auto g = loss_fn.backward();
            for (int c = 0; c < classes; ++c)
                grad[row * classes + c] = g[c] / (float)B;
        }
        model.backward(grad);
        optimizer.step();
    }
}
```

Doğrulama kopyası aynı dosyadadır. Derlemek için `NEXUS_MODEL_BUILD_EXAMPLES=ON`. Çalıştırılan ikili: `build-e2e/Release/train_end_to_end.exe`.

Checkpoint'i eğitime bağlamak

```
save_state_dict(model.state_dict(), "epoch_son.nxm");  
// sonra  
model.load_state_dict(load_state_dict_file("epoch_son.nxm"));  
model.eval();
```

Yüklemeden sonra optimizier durumunu ayrıca saklamazsınız; NexusOptim'in kendi `state_dict`'i vardır ve bu kılavuzun konusu değildir. Yalnızca ağırlıklar `NXM1` dosyasındadır. BatchNorm running istatistikleri buffer olduğu için onlar da yazılır.

Küçük bir conv ağı

```
Sequential cnn({  
    std::make_shared<Conv2D>(1, 8, 3, 1, 1),  
    std::make_shared<ReLU>(),  
    std::make_shared<MaxPool2D>(2),  
    std::make_shared<Conv2D>(8, 16, 3, 1, 1),  
    std::make_shared<ReLU>(),  
    std::make_shared<GlobalAvgPool2D>(),  
    std::make_shared<Linear>(16, 10)  
});  
cnn.train();  
Tensor images = Tensor::zeros({32, 1, 28, 28});  
Tensor logits = cnn.forward(images);    // [32, 10]
```

25 Performans ve optimizasyon

Ölçüm: 2026-09-29. AMD Ryzen 7 5800H with Radeon Graphics. Float32, tek hesaplama iş parçacığı, Release.

Linear 6×16 register blokları ve 8 KiB ağırlık panelleri kullanır. Paketleme ölçüme dahildir. Geri geçiş geçici transpoz oluşturmaz; convolution aynı motoru paylaşır. LayerNorm iki geçişli float64 istatistik hesabını SIMD ile yürütür.

Eski / yeni sürüm

Arşivlenmiş kaynak ve yeni sürüm aynı C++ ölçüm koduyla derlendi. Üç turda sürüm sırası değiştirildi; her tur 20 ısınma ve 30×5 çağrı. Aşağıdaki değerler 90 örneğin medyanıdır.

Çekirdek	Eski ms	Yeni ms	Oran
linear_forward_128_512_512	5.5838	1.8332	3.05×
linear_backward_128_512_512	14.2197	4.0230	3.53×
softmax_128_512	0.1421	0.0877	1.62×
layernorm_128_512	0.3115	0.0915	3.40×

NumPy ve PyTorch karşılaştırması

Üç turda kütüphane sırası değiştirilir. Aynı float32 girdiler, önceden ayrılmış çıktılar. NexusModel C++ çağrı maliyeti; NumPy/PyTorch Python çağrı maliyeti dahildir. Her çıktı float64 referansı ile doğrulanır. Süreler ms.

İşlem / şekil	NexusModel	NumPy	PyTorch
GEMM 1 × 256 × 256	0.00602	0.01371	0.01938
GEMM 7 × 129 × 33	0.00394	0.01593	0.00729
GEMM 32 × 256 × 256	0.15463	0.19405	0.14213
GEMM 96 × 768 × 192	0.74106	0.89292	0.80820
GEMM 128 × 512 × 512	1.76533	1.90798	1.86163
GEMM 256 × 1024 × 1024	15.06220	13.73203	14.43817
ReLU 65,536	0.01809	0.10000	0.02898
ReLU 1,048,576	0.23882	1.38747	0.31128
ReLU 4,194,304	2.19863	5.54118	2.31279

NexusModel 7/9 iş yükünde en düşük medyanı verdi. Küçük işlerde Python çağrı maliyeti önemlidir. Bu sonuçlar bütün boyutlarda veya bütün modellerde dünya birinciliği kanıtı değildir.

Tam katman ölçümleri

20 ısınma, 30×5 çağrı, tek tur medyanı. ReLU ileri; diğerleri ileri + geri. Eski katman sonuçlarıyla doğrudan optimizasyon oranı hesaplanmaz.

Katman	Şekil	Medyan ms
Linear	$64 \times 784 \rightarrow 256$	2.18014
Linear · scalar	$32 \times 256 \rightarrow 256$	5.30394
Linear · SIMD	$32 \times 256 \rightarrow 256$	0.40047
ReLU · scalar	1×2^{20}	0.65276
ReLU · SIMD	1×2^{20}	0.62537
Conv2D	$8 \times 16 \times 28 \times 28 \cdot K32 \cdot k3$	34.52230
MultiHeadAttention	$4 \times 32 \times 64 \cdot H4$	0.68537

Ölçüm sınırları

Güç profili, sıcaklık, çekirdek yakınlığı ve OS zamanlaması sabitlenmedi. GPU sonuçları eski sürüm arşividir ve bu optimizasyonda yeniden ölçülmedi. Ham örnekler: docs/public/benchmark-comparison.json ve optimization-results.json. Kaynaklar, sürümler ve hash değerleri raporda bulunur.

```
cmake -S . -B build
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
python docs/benchmarks/compare_libraries.py
```

26 Testler ve ölçüm

Ek çekirdek testleri `tests/test_kernels.cpp` dosyasında; kuyruk boyutları, gradyan birikimi, gruplu convolution ve teyp büyümesini denetler. `tests/test_model.cpp` ise şu çalıştırılabilirde toplanır: `nexus_model_tests`. Kapsam:

- Linear için elle matris. Ağırlık ve girdi yukarıdaki 2×3 örneğidir. Çıkış, dW, db ve dX birebir beklenir.
- Conv2D için 3×3 görüntü ve köşegen 2×2 çekirdek. Çıkış {6, 8, 12, 14}.
- Merkezi sonlu fark, epsilon $1e-3$. Linear, aktivasyon, Sequential, Conv1D/2D/3D, havuz, LayerNorm, BatchNorm, GroupNorm, InstanceNorm, RNN, LSTM, GRU, attention, encoder, residual.
- Embedding: pad satırı sıfır, tekrar eden indeksin gradyanı 2.
- state_dict yuvarlak turu ve fazla anahtarın hatası.
- İkinci forward+backward'ta `allocation_count` artmaz. Dikkat: ölçümden önce dönen tensör kapsam dışına çıkmalıdır.
- SIMD linear ile skalar linear, mutlak fark $2e-3$ altında.

ReLU ailesinde $x = 0$ noktası sonlu farka girmez. Orada fonksiyonun türevi yoktur; analitik alttürev 0'dır. Sayısal fark 0.5 verir ve bu bir kütüphane hatası değildir.

LibTorch bu depoya bağlı değildir. Referans, elle kurulan küçük tensörler ve sonlu farktır. Tolerans lineer örnekte $1e-4$ civarı, doğrusal olmayan sonlu farkta $2e-3$ ile $8e-3$ arasındadır. Float32 merkezi farkın kendisi bu kadar gürültülüdür.

```
NexusModel\build\Release\nexus_model_tests.exe
// başarı: all nexus_model checks passed
```

27 Dosya haritası

Yol	İçerik
include/nexus_model/nexus_model.hpp	Tek include
core/tensor.hpp	Hizalı tensör
core/parameter.hpp	data, grad, requires_grad
core/module.hpp	Katman tabanı, ağaç, mod
core/initializer.hpp	Kaiming, Xavier, orthogonal
core/serialization.hpp	NXM1
core/micro_tape.hpp	Attention teypi
core/kernels.hpp	Çalışma zamanı çekirdek tablosu, conv/pool/norm bildirimleri
core/error.hpp	ModelError
core/details.txt	Autograd kararı
layers/*.hpp	Katmanlar, başlık-only
activations/activations.hpp	Eleman bazlı aktivasyon
containers/	Sequential, ModuleList, ModuleDict
simd/	CPUID ve AVX bildirimleri
cuda/	Fused cihaz çekirdeği başlıkları
bridge/matrixflash.hpp	İsteğe bağlı GPU GEMM
adapters/nexus_optim.hpp	Parameter* → ParamGroup
src/core/scalar_kernels.cpp	Referans gövdeler
src/core/convolution.cpp, normalization.cpp, pooling.cpp, embedding.cpp, dropout.cpp	im2col, norm, pool, embedding
src/simd/avx2/	gemm.hpp, linear.cpp, normalization.cpp, details.txt
src/simd/	AVX2, AVX-512, atomik dağıtım
src/cuda/	.cu dosyaları, varsayılan kapalı
tests/test_model.cpp	Doğruluk
benchmarks/bench_suite.cpp	Süre
examples/train_end_to_end_example.cpp	Dört kütüphane birlikte
CMakeLists.txt	Seçenekler

Katman başlıkları kütüphaneye ayrıca derlenmez. Onları include ettiğiniz çeviri birimi, `nexus_model` kütüphanesindeki çekirdek sembollerine bağlanır. Bu yüzden her uygulama `target_link_libraries(... nexus_model)` demelidir.

28 Sınırlar ve hata mesajları

- Hesap float32'dir. FP16 ana kopya bu kütüphanede tutulmaz. Dar tip, NexusOptim'in master-copy sözleşmesine bırakılmıştır.
- Bir modül örneği eşzamanlı forward için güvenli değildir.
- Decoder, bellek gradyanını dönüş değerine eklemeyiz.
- RNN ailesi çift yönlü değildir ve gizli durumu çağrıdan çağıra taşımaz.
- ModuleList ve ModuleDict kendi başına forward çalıştırmaz.
- Conv3D çekirdek, stride, pad ve dilation için tek bir sayı alır; eksenler aynıdır.
- PositionalEncoding öğrenilen tabloyu, kurucudaki uzunluğun ötesine büyütmez.
- ResidualBlock çıkış eleman sayısını denetlemez.

ModelError	Ne zaman
Linear input last dimension must equal in_features	Son boyut yanlış
conv input rank mismatch / conv channel mismatch	NCHW düzeni bozuk
conv output spatial size is empty	Çekirdek, pad ve stride çıkışı yok ediyor
LayerNorm last dimension mismatch	normalized_size son boyuta eşit değil
embed must be divisible by num_heads	Kafa bölünmüyor
attention expects [B, T, E]	Rank 3 değil
causal attention requires equal query and key lengths	Nedensel maske kare değil
TransformerDecoderLayer requires forward(target, memory)	Bellek hiç verilmemiş
unexpected state_dict key / missing state_dict key	strict yükleme
state_dict shape mismatch for ...	Aynı isim, farklı şekil
state file magic mismatch	Dosya NXM1 değil
ModuleDict has no implicit forward	Sözlük kendi başına çalıştırıldı
reshape numel mismatch	-1 ile bile eleman sayısı tutmuyor

Sessiz öğrenememe

Yanlış backward, derlenir ve loss düşmeden durur. Şüphede `tests/test_model.cpp` içindeki sonlu fark düzenini kendi katmanınıza kopyalayın: parametreyi $\pm \epsilon$ oynatın, kaybın veya çıkış toplamının farkını 2ϵ 'a bölün, analitik gradyanla karşılaştırın. Linear gibi kapalı formlar $1e-4$ 'e iner. Tanh, GELU ve LSTM için $1e-3$ ile birkaç binde tolerans bekleyin.

Bu kılavuz NexusModel kaynak ağacındaki imzalarla yazıldı. Taslak sözde koddaki `MatrixFlash::Tensor`, `dataset.shuffle()` ve toplu `CrossEntropyLoss(prediction, labels)` bu depoda yoktur.